UNIVERSIDADE DE SÃO PAULO – ESCOLA POLITÉCNICA
PSI – MICROCONTROLADORES 2
Artur Serejo Dantas – Nº USP 15441730 – Turma B2-09
Atividade individual
Repositório: https://github.com/arturserejodantas7/psi-threads-padaria-filosofos

THREADS E RECURSOS COMPARTILHADOS: O PROBLEMA DA PADARIA

Parte 1 – Sem sincronização. O saldo da vitrine nem sempre representa a realidade. Com o padeiro produzindo a cada 1 s e o cliente consumindo a cada 1,5 s, o padeiro “ganha a corrida”: sobra cerca de um pão a cada 3 s, e depois de uns 30 s o saldo passa de 10, impossível numa vitrine de 10 lugares. Com esses tempos o saldo não fica negativo, mas o código não impede isso. Saldo negativo significaria vender um pão que não existe. A causa é que “saldo_vitrine++” parece uma ação só, mas o processador faz três passos: lê o valor, soma 1 e grava. Se a outra thread entra no meio desses passos (por exemplo, a cada 3 s as duas acordam juntas), as duas leem o mesmo valor e uma alteração se perde. Com pausas longas isso é raro, mas pode acontecer.

Parte 2 – Mudando os tempos. Padeiro a cada 500 ms e cliente a cada 2 s: entram 2 pães por segundo e sai meio, então o saldo cresce sem parar e estoura o limite de 10 em poucos segundos; o cliente sempre consegue pão. No caso contrário (padeiro 2 s, cliente 500 ms), o saldo cai rápido e fica negativo logo no começo, e o cliente continua “pegando” pão sem estoque.

Parte 3 – Prioridades. Com o padeiro mais prioritário, quando os dois ficam prontos ao mesmo tempo ele sempre roda primeiro, e a ordem das mensagens fica mais previsível. Com o cliente mais prioritário, ele consome primeiro e o saldo pode ficar negativo mais cedo. Em nenhum caso o problema desaparece: prioridade só decide quem roda primeiro, não protege a variável nem respeita o limite da vitrine.

Parte 4 – Como corrigir. No Zephyr, um mutex garante que só uma thread mexe no saldo por vez. Dois semáforos contadores resolvem os limites: um conta lugares vazios (começa em 10) e outro conta pães (começa em 0). Uma fila (k_msgq) de 10 posições faz tudo sozinha. A diferença entre semáforo binário e mutex é que o semáforo é um sinal, que qualquer thread pode liberar (serve para avisar “tem pão”), enquanto o mutex é uma chave com dono: só quem trancou pode destrancar. Para proteger o saldo, o mutex é o mais adequado.

REFERÊNCIA

ZEPHYR PROJECT. **Zephyr Project Documentation: kernel services.** Disponível em: https://docs.zephyrproject.org. Acesso em: 9 nov. 2026.

UNIVERSIDADE DE SÃO PAULO – ESCOLA POLITÉCNICA
PSI – MICROCONTROLADORES 2
Artur Serejo Dantas – Nº USP 15441730 – Turma B2-09
Atividade em grupo (realizada individualmente)

SOLUÇÃO PARA O PROBLEMA DO JANTAR DOS FILÓSOFOS

O problema. Cinco filósofos sentam numa mesa redonda e alternam entre pensar e comer. Entre cada par de vizinhos há um garfo, totalizando cinco, e cada um precisa dos dois garfos ao seu lado para comer. Se todos pegarem o garfo da esquerda ao mesmo tempo, todos ficam esperando o da direita para sempre: é o impasse (deadlock). Outro risco é a inanição, quando um filósofo quase nunca consegue comer.

Solução adotada. Usamos uma ordem fixa para pegar os garfos. Os garfos são numerados de 0 a 4 e cada filósofo sempre pega primeiro o garfo de menor número e depois o de maior. Para quatro filósofos isso não muda nada, mas o último (que fica entre os garfos 4 e 0) passa a pegar o garfo 0 primeiro. Com isso, a “roda” de espera é quebrada: é impossível que todos segurem um garfo e esperem o próximo ao mesmo tempo, então sempre existe alguém que consegue comer e depois libera os garfos para os vizinhos.

Implementação no Zephyr. Cada garfo é um k_mutex e cada filósofo é uma thread, todas com a mesma prioridade. O laço de cada thread é: pensar (k_msleep com um tempo diferente para cada filósofo), travar o garfo de menor número, travar o de maior número, comer (k_msleep), destravar os dois e voltar a pensar. O mutex garante que um garfo nunca está na mão de dois filósofos. O código está na pasta filosofos do repositório.

Alternativas consideradas. (a) Garçom: um semáforo contador iniciado em 4 deixa no máximo quatro filósofos tentarem comer de uma vez, então pelo menos um sempre consegue os dois garfos. (b) Tentar e devolver: o filósofo tenta pegar o segundo garfo sem esperar (K_NO_WAIT) e, se não conseguir, devolve o primeiro; evita o impasse, mas pode gerar uma situação em que todos ficam pegando e soltando sem comer. Escolhemos a ordem fixa por ser simples, não precisar de recurso extra e eliminar o impasse na origem.

REFERÊNCIA

DIJKSTRA, E. W. Hierarchical ordering of sequential processes. **Acta Informatica**, v. 1, n. 2, p. 115-138, 1971.